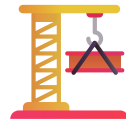


Structures as Function Arguments



Understanding user-defined data types in C programming

Structures as Function Arguments

C is a structured programming language and supports the basic concept in it is the modularity of programs. This concept is supported by functions in C language. Let us look into techniques of passing structures to functions. This can be achieved in primarily two ways: Firstly, to pass them as simple parameter values by passing the structure name and secondly, through pointers. We will be concentrating on the first method in this unit and passing using pointers will be taken up in the next unit.

Like other data types, a structure can be passed as an argument to a function. The program listing given below shows how to do this.



Example Program

```
#include<stdio.h>
#include<string.h>
struct Books
```

```

{
    char title[50];
    char author[50];
    char subject[100];
    int book_id;
};
/* function declaration */
void printBook( struct Books /* Declare Book1 of type Book */ );
int main( )
{
    struct Books Book1; /* Declare Book1 of type Book */
    struct Books Book2; /* Declare Book2 of type Book */
    /* book 1 specification */
    strcpy( Book1.title, "C Programming");
    strcpy( Book1.author, "Nuha Ali");
    strcpy( Book1.subject, "C Programming Tutorial");
    Book1.book_id = 6495407;
    /* book 2 specification */
    strcpy( Book2.title, "Telecom Billing");
    strcpy( Book2.author, "Zara Ali");
    strcpy( Book2.subject, "Telecom Billing Tutorial");
    Book2.book_id = 6495700;
    /* print Book1 info */
    printBook( Book1 );
    /* Print Book2 info */
    printBook( Book2 );
    return 0;
}

```

Initializing Structures

C variable types, structures can be initialized when they're declared. This procedure is similar to that for initializing arrays. The structure declaration is followed by an equal sign and a list of initialization values is separated by commas and enclosed in braces.

Example:



Customer and Sale Structure Example

```
struct customer
{
    char firm[20];
    char contact[25];
}
struct sale
{
    struct customer buyer1;
    char item[20];
    float amt;
```

```
}  
mysale = { "XYZ Industries", "Tyran Adams", "toolkit", 600.00 };
```

These statements perform the following initializations:



Firm Initialization

structure member `mysale.buyer1.firm` is initialized to string "XYZ Industries"



Contact Initialization

structure member `mysale.buyer1.contact` is initialized to string "Tyran Adams"



Item Initialization

structure member `mysale.item` is initialized to string "toolkit"



Amount Initialization

structure member `mysale.amount` is initialized to amount 600.00

When initializing a structure variable, you can assign values to individual members or initialize the entire

structure at once.

8

Pointers to Structures

To define pointers to structures in the same way as you define pointer to any other variable:

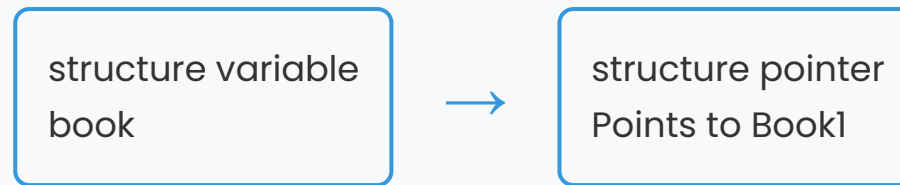


Structure Pointer Declaration

```
struct Books *struct_pointer;
```

Now, you can store the address of a structure variable in the above-defined pointer variable. To

find the address of a structure variable, place the '&' operator before the structure's name as:



To access the members of a structure using a pointer to that structure, to operator as:



Accessing Structure Members via Pointer

```
struct_pointer->title;
/* Accesses title member */
printf( "Book title : %s\n", struct_pointer->title);
printf( "Book author : %s\n", struct_pointer->author);
printf( "Book subject : %s\n", struct_pointer->subject);
printf( "Book book_id : %d\n", struct_pointer->book_id);
```

Let us rewrite the example using structure pointer:



Rewritten Example Using Pointers

```
#include<stdio.h>
#include<string.h>
struct Books
{
    char title[50];
    char author[50];
    char subject[100];
    int book_id;
};
/* function declaration */
void printBook( struct Books /* Declare Book1 of type Book */ );
struct Books Book2; /* Declare Book2 of type Book */
/* book 1 specification */
strcpy( Book1.title, "C Programming");
strcpy( Book1.author, "Nuha Ali");
strcpy( Book1.subject, "C Programming Tutorial");
Book1.book_id = 6495407;
/* book 2 specification */
strcpy( Book2.title, "Telecom Billing");
strcpy( Book2.author, "Zara Ali");
strcpy( Book2.subject, "Telecom Billing Tutorial");
Book2.book_id = 6495700;
/* print Book1 info by passing address of Book1 */
printBook( &Book1 );
/* print Book2 info by passing address of Book2 */
```



```
printBook( &Book2 );  
return 0;  
}
```

Conclusion

Key Takeaways



Data Organization

Structures allow grouping related data into a single entity



Flexible Design

Can hold multiple data types in one place



Member Access

Accessed using dot (.) operator



Pointer Support

Can pass structures to functions by reference using pointers

Best Practices



Initialization

Initialize structure members at declaration or using dot notation



Memory Management

Use pointers for efficient data manipulation

Structures are fundamental to creating complex data structures in C programming, essential for building databases, file systems, and many real-world applications.

